

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



Android Layout With Compose

Oleh:

Rika Fauliana Rahmi NIM. 2410817120017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rika Fauliana Rahmi
NIM : 2410817120017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	8
B. Output Program	15
C. Pembahasan	19
D. Tautan Git	21

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi.....	6
Gambar 2. Tampilan Pilihan Persentase Tip	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan	7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML.....	16
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML.....	16
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML.....	17
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose	17
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose	18
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose	18

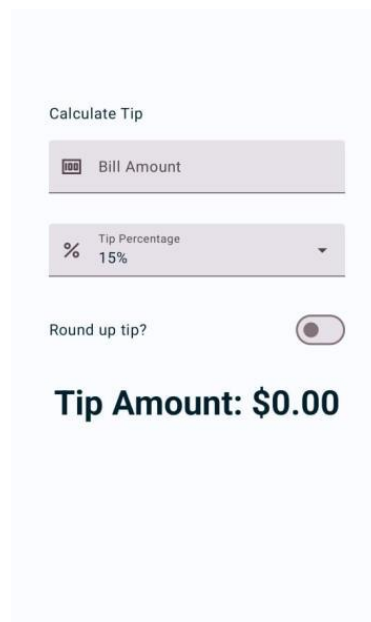
DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML Soal 1	8
Tabel 2. Source Code activity_main.xml Soal 1	10
Tabel 3. Source Code MainActivity.kt Compose Soal 1.....	12

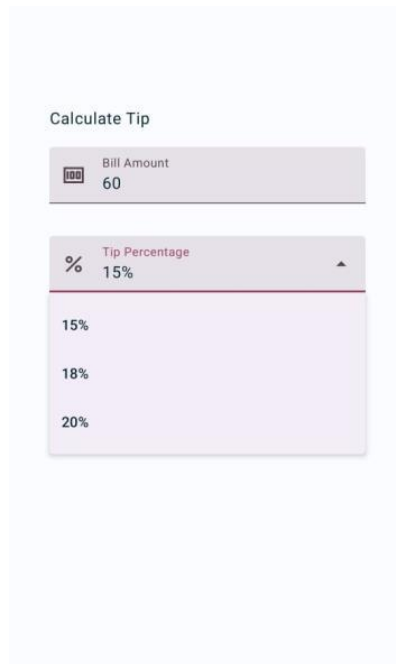
SOAL 1

Soal Praktikum:

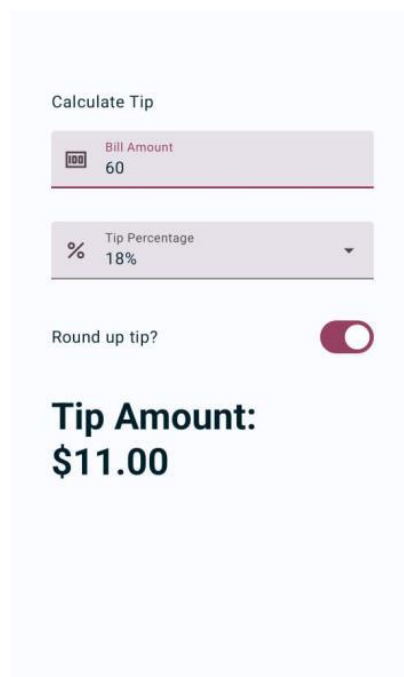
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
 - a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
 - b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
 - c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
 - d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.
2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

Additional:

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Source Code

MainActivity.kt XML

Tabel 1. Source Code MainActivity.kt XML Soal 1

1	package com.example.modul2xml
2	
3	import android.os.Bundle
4	import android.text.Editable
5	import android.text.TextWatcher
6	import android.widget.AdapterView
7	import android.widget.AutoCompleteTextView
8	import android.widget.TextView
9	import androidx.appcompat.app.AppCompatActivity
10	import
	com.google.android.material.materialswitch.MaterialSwitch
	ch
11	import
	com.google.android.material.textfield.TextInputEditText
12	import java.util.Locale
13	import kotlin.math.ceil
14	
15	class MainActivity : AppCompatActivity() {
16	
17	private lateinit var etBillAmount: TextInputEditText
18	private lateinit var spinnerTipPercentage: AutoCompleteTextView
19	private lateinit var switchRoundUp: MaterialSwitch
20	private lateinit var tvTipResult: TextView
21	
22	private var currentPercentage = 15
23	
24	override fun onCreate(savedInstanceState: Bundle?) {
25	super.onCreate(savedInstanceState)
26	setContentView(R.layout.activity_main)
27	
28	etBillAmount = findViewById(R.id.etBillAmount)
29	spinnerTipPercentage =
	findViewById(R.id.spinnerTipPercentage)
30	switchRoundUp = findViewById(R.id.switchRoundUp)


```

31         tvTipResult = findViewById(R.id.tvTipResult)
32
33         setupDropdown()
34         setupListeners()
35     }
36
37     private fun setupDropdown() {
38         val options = arrayOf("15%", "18%", "20%")
39         val adapter = ArrayAdapter(this,
40 android.R.layout.simple_list_item_1, options)
41         spinnerTipPercentage.setAdapter(adapter)
42         spinnerTipPercentage.setOnItemClickListener { _,
43 _, position, _ ->
44             currentPercentage = when (position) {
45                 0 -> 15
46                 1 -> 18
47                 2 -> 20
48                 else -> 15
49             }
50             calculateTip()
51         }
52
53     private fun setupListeners() {
54         etBillAmount.addTextChangedListener(object :
55 TextWatcher {
56             override fun beforeTextChanged(s:
57 CharSequence?, start: Int, count: Int, after: Int) {}
58             override fun onTextChanged(s: CharSequence?,
59 start: Int, before: Int, count: Int) {}
60             override fun afterTextChanged(s: Editable?)
61 {
62                 calculateTip()
63             }
64         })
65     }
66
67     switchRoundUp.setOnCheckedChangeListener { _, _
68 ->
69         calculateTip()
70     }
71
72     private fun calculateTip() {
73         val amountStr = etBillAmount.text.toString()
74         val amount = amountStr.toDoubleOrNull() ?: 0.0

```

71	var tip = amount * (currentPercentage / 100.0)
72	
73	if (switchRoundUp.isChecked) {
74	tip = ceil(tip)
75	}
76	
77	tvTipResult.text = String.format(Locale.US, "Tip
78	Amount: \$%.2f", tip)
79	}
79	}

activity_main.xml

Tabel 2. Source Code activity_main.xml Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:paddingHorizontal="32dp"
8	android:paddingVertical="48dp"
9	android:background="#FBFAFE"> <TextView
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:text="Calculate Tip"
13	android:textSize="16sp"
14	android:textStyle="bold"
15	android:textColor="#4A4458"
16	android:layout_marginBottom="24dp"/>
17	
18	<com.google.android.material.textfield.TextInputLayout
19	android:layout_width="match_parent"
20	android:layout_height="wrap_content"
21	android:hint="Bill Amount"
22	app:boxBackgroundColor="#E8E2EC"
23	app:boxStrokeColor="#6750A4"
24	app:hintTextColor="#6750A4"
25	app:boxCornerRadiusTopStart="4dp"
26	app:boxCornerRadiusTopEnd="4dp"
27	android:layout_marginBottom="24dp">
28	

```

29 <com.google.android.material.textfield.TextInputEditText
30     android:id="@+id/etBillAmount"
31     android:layout_width="match_parent"
32     android:layout_height="wrap_content"
33     android:inputType="numberDecimal"
34     android:textColor="#1D1B20"
35     android:singleLine="true"/>
36
37 </com.google.android.material.textfield.TextInputLayout
38 >
39 <com.google.android.material.textfield.TextInputLayout
40 style="@style/Widget.MaterialComponents.TextInputLayout
41 .FilledBox.ExposedDropdownMenu"
42     android:layout_width="match_parent"
43     android:layout_height="wrap_content"
44     android:hint="Tip Percentage"
45     app:boxBackgroundColor="#E8E2EC"
46     app:boxStrokeColor="#6750A4"
47     app:hintTextColor="#6750A4"
48     app:boxCornerRadiusTopStart="4dp"
49     app:boxCornerRadiusTopEnd="4dp"
50     android:layout_marginBottom="24dp">
51     <AutoCompleteTextView
52         android:id="@+id/spinnerTipPercentage"
53         android:layout_width="match_parent"
54         android:layout_height="wrap_content"
55         android:inputType="none"
56         android:textColor="#1D1B20"
57         android:text="15%"
58         android:popupBackground="#E8E2EC"/>
59
60 </com.google.android.material.textfield.TextInputLayout
61 >
62 <LinearLayout
63     android:layout_width="match_parent"
64     android:layout_height="48dp"
65     android:orientation="horizontal"
66     android:gravity="center_vertical"
67     android:layout_marginBottom="24dp">

```

67	<TextView
68	android:layout_width="0dp"
69	android:layout_height="wrap_content"
70	android:layout_weight="1"
71	android:text="Round up tip?"
72	android:textSize="16sp"
73	android:textColor="#1D1B20"/>
74	
75	<com.google.android.material.materialswitch.MaterialSwitch
	tch
76	android:id="@+id/switchRoundUp"
77	android:layout_width="wrap_content"
78	android:layout_height="wrap_content" />
79	</LinearLayout>
80	
81	<TextView
82	android:id="@+id/tvTipResult"
83	android:layout_width="match_parent"
84	android:layout_height="wrap_content"
85	android:text="Tip Amount: \$0.00"
86	android:textSize="32sp"
87	android:textStyle="bold"
88	android:textColor="#001C38"
89	android:layout_marginTop="16dp"/>
90	
91	</LinearLayout>

MainActivity.kt Compose

Tabel 3. Source Code MainActivity.kt Compose Soal 1

1	package com.example.modul2compose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.foundation.rememberScrollState
8	import androidx.compose.foundation.text.KeyboardOptions
9	import androidx.compose.foundation.verticalScroll
10	import androidx.compose.material3.*
11	import androidx.compose.runtime.*
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.text.font.FontWeight
15	import androidx.compose.ui.text.input.KeyboardType
16	import androidx.compose.ui.unit.dp

```

17 import androidx.compose.ui.unit.sp
18 import java.util.Locale
19 import kotlin.math.ceil
20
21 class MainActivity : ComponentActivity() {
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24         setContent {
25             TipCalculatorApp()
26         }
27     }
28 }
29
30 @OptIn(ExperimentalMaterial3Api::class)
31 @Composable
32 fun TipCalculatorApp() {
33     var amountInput by remember { mutableStateOf("") }
34     var tipPercentage by remember {
35         mutableIntStateOf(15) }
36     var roundUp by remember { mutableStateOf(false) }
37     var expanded by remember { mutableStateOf(false) }
38
39     val options = listOf(15, 18, 20)
40
41     val amount = amountInput.toDoubleOrNull() ?: 0.0
42     var tip = amount * (tipPercentage / 100.0)
43     if (roundUp) {
44         tip = ceil(tip)
45     }
46
47     Column(
48         modifier = Modifier
49             .padding(horizontal = 32.dp, vertical =
50 48.dp)
51             .fillMaxSize()
52             .verticalScroll(rememberScrollState()),
53         verticalArrangement =
54 Arrangement.spacedBy(24.dp)
55     ) {
56         Text(
57             text = "Calculate Tip",
58             fontSize = 16.sp,
59             fontWeight = FontWeight.Medium
60         )
61
62         TextField(
63             value = amountInput,

```

```

61         onChange = { amountInput = it },
62         label = { Text("Bill Amount") },
63         keyboardOptions                                =
KeyboardOptions(keyboardType = KeyboardType.Number),
64         singleLine = true,
65         modifier = Modifier.fillMaxWidth()
66     )
67
68     ExposedDropDownMenuBox(
69         expanded = expanded,
70         onExpandedChange = { expanded = !expanded }
71     ) {
72         TextField(
73             value = "$tipPercentage%",
74             onChange = {},
75             readOnly = true,
76             label = { Text("Tip Percentage") },
77             trailingIcon                                = {
ExposedDropDownMenuDefaults.TrailingIcon(expanded
expanded) },
78             colors                                      =
ExposedDropDownMenuDefaults.textFieldColors(),
79             modifier = Modifier
80
.menuAnchor(MenuAnchorType.PrimaryNotEditable)
81                 .fillMaxWidth()
82         )
83         ExposedDropDownMenu(
84             expanded = expanded,
85             onDismissRequest = { expanded = false }
86         ) {
87             options.forEach { selectionOption ->
88                 DropdownMenuItem(
89                     text                                = {
Text("$selectionOption%") },
90                     onClick = {
91                         tipPercentage                    =
selectionOption
92                         expanded = false
93                     }
94                 )
95             }
96         }
97     }
98
99     Row(
100         modifier = Modifier

```

101	.fillMaxWidth()	
102	.size(48.dp),	
103	verticalAlignment	=
	Alignment.CenterVertically,	
104	horizontalArrangement	=
	Arrangement.SpaceBetween	
105	) {	
106	Text("Round up tip?")	
107	Switch(	
108	checked = roundUp,	
109	onCheckedChange = { roundUp = it }	
110	)	
111	}	
112		
113	Column(modifier = Modifier.padding(top = 16.dp))	
	{	
114	Text(	
115	text = "Tip Amount:",	
116	fontSize = 36.sp,	
117	fontWeight = FontWeight.Bold	
118	)	
119	Text(	
120	text = "\${String.format(Locale.US,	
	"%0.2f", tip)}",	
121	fontSize = 36.sp,	
122	fontWeight = FontWeight.Bold	
123	)	
124	}	
125	}	
126	}	

B. Output Program

XML:

Calculate Tip

Bill Amount

Tip Percentage
15%

Round up tip? ☐

Tip Amount: \$0.00

Gambar 4. Screenshot Hasil Jawaban Soal 1 XML

Calculate Tip

Bill Amount
60

Tip Percentage
15%

15%

18%

20%

1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p

Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

Calculate Tip

Bill Amount
60

Tip Percentage
18%

Round up tip? ☒

Tip Amount: \$11.00

Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

Compose

Calculate Tip

Bill Amount

Tip Percentage
15%

Round up tip? ☐

**Tip Amount:
\$0.00**

Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose

Calculate Tip

Bill Amount
60

Tip Percentage
15%

15%

18%

20%

\$9.00

Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose

Calculate Tip

Bill Amount
60

Tip Percentage
18%

Round up tip? ☒

**Tip Amount:
\$11.00**

Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt XML

Pada file MainActivity.kt, logika aplikasi mulai berjalan di dalam fungsi 'onCreate()'. Fungsi 'setContentView(R.layout.activity_main)' dipanggil untuk menghubungkan class Kotlin ini dengan desain XML yang telah dibuat. Lalu, dilakukan proses inisialisasi variabel menggunakan 'findViewById' untuk menghubungkan komponen UI seperti kolom input tagihan, menu dropdown persentase, switch pembulatan, dan teks hasil dengan variabel di dalam kode Kotlin. Setelah inisialisasi, program memanggil fungsi 'setupDropdown()' dan 'setupListeners()' untuk mempersiapkan interaksi pengguna.

Di dalam fungsi 'setupDropdown()', opsi "15%", "18%", dan "20%" dimasukkan ke dalam menu dropdown menggunakan ArrayAdapter. Ketika pengguna memilih salah satu opsi tersebut, blok 'setOnClickListener' akan menangkap urutan pilihannya, memperbarui nilai persentase menggunakan struktur percabangan when, dan langsung memanggil fungsi 'calculateTip()'. Sementara itu, di dalam fungsi 'setupListeners()', program menambahkan 'TextWatcher' pada kolom input agar setiap ketikan baru memicu aksi 'afterTextChanged', dan menambahkan 'setCheckedChangeListener' pada tombol switch. Kedua listener ini memastikan bahwa setiap perubahan ketikan atau geseran tombol akan langsung memanggil fungsi 'calculateTip()' secara otomatis.

Aksi utama perhitungannya terjadi di dalam fungsi 'calculateTip()'. Program mengambil teks tagihan yang diketik pengguna, mengubahnya menjadi tipe desimal, dan menghitung nominal tip dari perkalian tagihan dengan persentase yang dipilih. Setelah perhitungan dasar didapatkan, program menggunakan percabangan if untuk mengecek status tombol switch; jika switch menyala 'isChecked', nilai tip akan dibulatkan ke bilangan bulat terdekat ke atas menggunakan fungsi 'ceil()'. Terakhir, program menggunakan 'String.format' untuk mengatur nilai akhir menjadi format mata uang dengan dua angka desimal, lalu memperbarui teks pada layar untuk menampilkan hasilnya.

activity_main.xml

Pada file activity_main.xml, antarmuka pengguna (UI) dibangun menggunakan komponen 'LinearLayout' sebagai wadah utama dengan orientasi vertikal, yang menyusun semua elemen secara berurutan dari atas ke bawah. Latar belakang layar ini diatur

menggunakan warna abu-abu keunguan pudar yang merupakan warna khas Material 3, dilengkapi dengan padding horizontal dan vertikal agar tampilan memiliki ruang dan tidak menempel pada pinggir layar. Elemen pertama yang dimuat di dalam wadah ini adalah sebuah ‘TextView’ yang berfungsi sebagai judul halaman bertuliskan "Calculate Tip" dengan format teks tebal.

Selanjutnya, tata letak menampilkan kolom input untuk nominal tagihan yang dibungkus menggunakan ‘TextInputLayout’ dari pustaka Material Design. Komponen pembungkus ini memberikan efek visual modern seperti warna latar kotak ‘boxBackgroundColor’ dan garis tepi ‘boxStrokeColor’, sementara di dalamnya disematkan ‘TextInputEditText’ ber-ID ‘etBillAmount’ yang secara spesifik diatur agar hanya menerima ketikan angka desimal. Tepat di bawahnya, terdapat menu dropdown untuk memilih persentase tip yang juga dibungkus oleh ‘TextInputLayout’, namun diberikan gaya tambahan ‘ExposedDropdownMenu’ agar berfungsi sebagai menu tarik-turun. Di dalam bungkus tersebut, diletakkan ‘AutoCompleteTextView’ ber-ID ‘spinnerTipPercentage’ yang menampilkan teks bawaan "15%" dan memiliki atribut ‘popupBackground’ agar warna kotak menunya selaras saat diklik.

Untuk fitur pembulatan angka, program menyisipkan wadah ‘LinearLayout’ baru yang berorientasi horizontal agar elemen di dalamnya tersusun menyamping. Di dalam wadah penyambung ini, sebuah ‘TextView’ yang berisi teks "Round up tip?" dibentangkan menggunakan ‘layout_weight’ agar mendorong dan mensejajarkan komponen ‘MaterialSwitch’ ber-ID ‘switchRoundUp’ tepat di ujung kanan layar sebagai tombol sakelarnya. Sebagai penutup antarmuka, tata letak diakhiri dengan sebuah ‘TextView’ ber-ID ‘tvTipResult’. Komponen penampil hasil ini diatur dengan ukuran huruf yang besar, ditebalkan, dan diberi warna kontras untuk menonjolkan fungsinya sebagai tempat menampilkan teks hasil akhir perhitungan yang secara bawaan bertuliskan "Tip Amount: \$0.00".

MainActivity.kt Compose

Pada file MainActivity.kt, aplikasi dijalankan melalui fungsi ‘setContent’ yang memuat composable utama bernama ‘TipCalculatorApp()’. Di dalamnya, program mendeklarasikan beberapa variabel state menggunakan ‘remember’ dan ‘mutableStateOf’,

seperti 'amountInput' untuk teks tagihan, 'tipPercentage' untuk nilai tip, 'roundUp' untuk status sakelar pembulatan, dan 'expanded' untuk status menu dropdown. Program langsung memuat logika perhitungan dengan mengonversi 'amountInput' menjadi angka, menghitung nilai 'tip', dan menerapkan fungsi 'ceil()' jika 'roundUp' bernilai true. Antarmuka utama dibungkus oleh komponen 'Column' berserta 'Modifier' seperti 'verticalScroll' agar layar bisa digulir. Di dalam 'Column' ini, dimuat komponen 'Text' sebagai judul dan 'TextField' sebagai kolom input tagihan yang diatur dengan 'KeyboardOptions' agar hanya memunculkan keyboard angka.

Untuk bagian menu persentase tip, tata letak menggunakan 'ExposedDropdownMenuBox'. Di dalamnya, terdapat sebuah 'TextField' yang diatur 'readOnly' untuk berfungsi sebagai tombol jangkar. Saat kolom ini diklik, nilai state 'expanded' akan berubah dan memunculkan 'ExposedDropdownMenu'. Menu ini menggunakan perulangan 'forEach' untuk menampilkan deretan opsi 15%, 18%, dan 20% ke dalam bentuk 'DropdownMenuItem' yang bisa dipilih pengguna.

Pada bagian fitur pembulatan, program menggunakan tata letak 'Row' dengan pengaturan 'SpaceBetween' agar teks "Round up tip?" berada di sisi kiri dan komponen 'Switch' terdorong ke ujung kanan layar. Komponen 'Switch' ini dihubungkan langsung dengan state 'roundUp' sehingga setiap perubahan langsung memicu perhitungan ulang. Sebagai penutup, di bagian paling bawah terdapat 'Column' tambahan yang memuat dua komponen 'Text' berukuran besar. 'Text' yang terakhir secara khusus bertugas menampilkan nilai perhitungan akhir yang telah diformat menjadi dua angka desimal menggunakan fungsi 'String.format'.

D. Tautan Git

<https://github.com/rikafaulianarahmi/MODUL-MOBILE/tree/main/MODUL%202>

Additional:

1. Fitur yang menjadi daya jual Kotlin:

- Null Safety, secara default kotlin membedakan antara referensi yang boleh dan yang tidak boleh bernilai null. Fitur ini bertujuan untuk menghilangkan bahaya

‘NullPointerException’ dari kode, yang disebut sebagai “The Billion Dollar Mistake” (Jemerov & Isakova, 2017).

- Interoperabilitas penuh dengan Java, Kotlin dirancang untuk dapat bekerja berdampingan dengan Java. Pengembang dapat memanggil class Java dari dalam kode Kotlin, sehingga memudahkan transisi aplikasi lama ke teknologi baru secara bertahap (Jemerov & Isakova, 2017).
- Conciseness, Kotlin memungkinkan penulisan kode yang jauh lebih pendek dibandingkan Java untuk fungsi yang sama. Berdasarkan penelitian perbandingan, penggunaan Kotlin secara signifikan mengurangi jumlah baris kode, yang secara langsung mempermudah pemeliharaan dan pembacaan program (Sulowski & Koziel, 2019).
- Coroutines, Fitur ini menawarkan solusi untuk pemrograman asinkron yang efisien. Coroutines memungkinkan pengelolaan proses berat (seperti akses database atau API) berjalan di latar belakang tanpa menghambat antarmuka pengguna, sehingga performa aplikasi tetap lancar (Sulowski & Koziel, 2019).
- Extension Functions, menambahkan fungsi-fungsi baru ke class yang sudah ada tanpa harus melakukan inheritance (Jemerov & Isakova, 2017).

2. Test case untuk fitur Null Safety menggunakan operator Safe Call (?.) dan Elvis Operator (?:).

Sebuah fungsi bernama ‘getMemberStatus’ menerima input nama anggota yang bisa bernilai null dan mengembalikan pesan status. Logika dari fungsi ini yaitu jika nama ada, maka tampilkan nama tersebut, namun jika nama kosong (null), maka tampilkan “Tamu”.

```
package com.example.testcasemodul2

import org.junit.Assert.assertEquals
import org.junit.Test

class NullSafetyTest {
    private fun getMemberStatus(name: String?): String {
        return name ?: "Tamu"
    }

    @Test
    fun testValidasiInputNamaTersedia() {
```

```

        val inputName = "Budi"
        val expectedResult = "Budi"
        val actualResult = getMemberStatus(inputName)
        assertEquals(expectedResult, actualResult)
    }

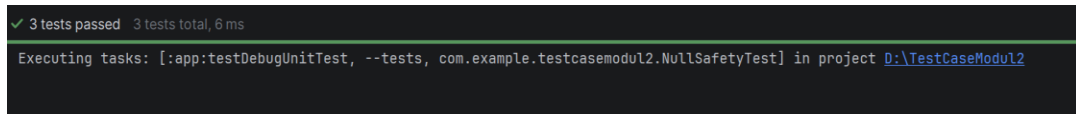
    @Test
    fun testValidasiInputNamaBernilaiNull() {
        val inputName: String? = null
        val expectedResult = "Tamu"
        val actualResult = getMemberStatus(inputName)
        assertEquals(expectedResult, actualResult)
    }

    @Test
    fun testValidasiInputNamaStringKosong() {
        val inputName = ""
        val expectedResult = ""
        val actualResult = getMemberStatus(inputName)
        assertEquals(expectedResult, actualResult)
    }
}

```

- Test Case 1: Validasi input nama tersedia
 - Data Input: 'name = "Budi"'
 - Langkah Pengujian: Masukkan string "Budi" ke dalam fungsi 'getMemberStatus', lalu jalankan fungsi tersebut.
 - Hasil yang Diharapkan: Sistem berhasil membaca input dan mengembalikan nilai "Budi".
 - Status: Berhasil (Pass).
- Test Case 2: Validasi input nama bernilai null
 - Data Input: name = null
 - Langkah Pengujian: Masukkan nilai null secara sengaja ke dalam fungsi 'getMemberStatus', lalu jalankan fungsi tersebut.
 - Hasil yang Diharapkan: Sistem tidak mengalami error (crash) atau NullPointerException. Sistem mendeteksi nilai null dan langsung mengembalikan nilai default "Tamu".
 - Status: Berhasil (Pass).
- Test Case 3: Validasi input berupa string kosong

- Data Input: name = ""
- Langkah Pengujian: Masukkan string kosong "" ke dalam fungsi getMemberStatus, lalu jalankan fungsi tersebut.
- Hasil yang Diharapkan: Sistem mengembalikan nilai string kosong "" (karena dalam Kotlin, string kosong dihitung sebagai objek riil yang eksis, bukan tipe data null).
- Status: Berhasil (Pass).



```

✓ 3 tests passed 3 tests total, 6 ms
Executing tasks: [:app:testDebugUnitTest, --tests, com.example.testcasemodul2.NullSafetyTest] in project D:\TestCaseModul2

```

Analisis Hasil:

Berdasarkan pengujian pada Test Case 2, terbukti bahwa penggunaan fitur Null Safety Kotlin menjamin aplikasi terhindar dari penghentian paksa (Force Close). Jika ini ditulis dalam bahasa Java tanpa pengecekan blok if-else yang panjang, memasukkan nilai null akan memicu Runtime Error. Pada Kotlin, keberadaan Elvis Operator (?:) sangat efektif memberikan penanganan yang ringkas dan aman terhadap variabel tak bernilai.

DAFTAR PUSTAKA

- Sulowski, D., & Koziel, G. (2019). Comparative analysis of Kotlin and Java languages used to create applications for the Android system. *Journal of Computer Sciences Institute*, 13, 354–358.
- Jemerov, D., & Isakova, S. (2017). *Kotlin in action*. Manning Publications.